

1 Question 4

1.1 AI Changes

First of all, I did this computer code in python, so I asked AI to change the code from the provided matlab code to python. Here were the differences the AI (Google Gemini because we get the pro version for free from NCSU) reported:

1. Slicing (2:end-1 vs 1:-1): Python uses 0-based indexing and exclusive upper bounds. The interior nodes 2:end-1 in MATLAB translate to 1:-1 in Python.
2. Meshgrid Orientation (indexing='ij'): Passing indexing='ij' to NumPy's meshgrid keeps coordinate matrix array structures aligned cleanly with standard row/column Cartesian indices.
3. Backslash Operator (\): MATLAB's $G \setminus U(:,n)$ is handled via `scipy.sparse.linalg.spsolve(G, U[:, n])` for sparse matrices.

Here is the exact prompt I used: "I am doing a numerical methods homework and have been provided a file in matlab. However, I prefer working in python. Take the attached file [I attached the provided *.m file] and transcribe it to python."

1.2 My Changes

Changes Applied to Solver: Next, I changed 'solve_heat_2d_implicit()' to be a function of a 'mode' parameter. The 'mode' parameter could be either "default", "factored", or "optimized" each corresponding to the method the solution was generated with.

- "default" corresponds to the default or "naive" implementation of the solution. This is the method provided in the given matlab file. In python, I used the function 'scipy.linalg.spsolve', which is the default method for sparse matrix solving. In matlab, the "default" mode corresponds to the base implementation of the backslash operator.
- "factored" corresponds to 'chol' in matlab. However, I am not using Cholesky factorization, but am instead using sparse LU factorization via 'scipy.linalg.splu(G, permc_spec="NATRUAL")' to generate the LU factors. Then, I am solving with 'scipy.linalg.splu(G).solve(U[:, n])'. Here, "permc_spec" is a parameter that specifies how the columns will be permuted in the factorization. For 'permc_spec="NATURAL"', the columns are ordered as they are built and undergo no permutation. This is effectively equivalent to 'chol' in matlab, which also factorizes based on the natural ordering. I have a more in-depth discussion about my use of LU factorization over Cholesky factorization in the "Asides" following this list.

- "optimized" corresponds to matlab's 'symamd'. However, I am not using optimized Cholesky factorization, but am instead using 'scipy.linalg.splu(G, permc_spec="MMD_AT_PLUS_A")'. Then, I am solving with 'scipy.linalg.splu(G).solve(U[:, n])'. 'symamd' stands for "SYMmetric Approximate Minimum Degree" and 'permc_spec="MMD_AT_PLUS_A"' stands for MiniMum Degree ordering of $A^T + A$. Both algorithms permute the factored matrices so the resulting factors (Cholesky and LU, respectively) maintain as much sparsity from the original matrix (G) as possible.

Aside about using Cholesky vs LU Factors: I argue the use of Cholesky or LU factorization here is irrelevant for what this problem is trying to demonstrate. The goal of this problem is to show how caching matrix factors (i.e, the Cholesky or LU factors) for a linear system improves the run-time AND how you can achieve even higher speed-up when you generate matrix factors with more sparsity (i.e., the permuted Cholesky and permuted LU factors). We are not implementing a Cholesky or LU factorizer, but just running a command in matlab or python. As will be shown in the results section, we see the same effect as expected from doing the Cholesky factorization.

Aside about Software Licensing: The only reason I am even justifying the use of a different factorization is because scipy does not have native support for a true sparse Cholesky routine because of copyright issues. CHOLMOD, the "gold-standard" Cholesky factorization algorithm, is distributed as part of the SuiteSparse toolkit. SuiteSparse has a GNU General Public License (GPL) which mandates and software that uses code from SuiteSparse must also be GPL. However, SciPy is distributed under the Berkeley Software Distribution Licence (BSD-3). As such, scipy cannot implement CHOLMOD natively. In python, you can use SuiteSparse, and subsequently CHOLMOD, by using the wrapper package "scikit-sparse". However, I am on Arch Linux, an experimental version of linux, and could not get scikit-sparse installed. I tried for a while and just decided to use the functions already in scipy.

Other Changes: The rest of the changes I made were just adding plotting to a snapshot of each mode every second from 0s, the initial time, to 10s, the final time. All other changes are too minor to worth noting. I added comments in § 2 to specifically show what I added compared to what the AI translated for me.

1.3 Results

Runtime: I gave these times to AI and told it to make a nicely formatted LaTeX table. Here is the exact prompt I used: "I am doing a homework and have the runtime outputs for different approaches for solving the 2D heat equation. Create a nicely formatted LaTeX table including the runtimes and speedup factors over the default method."

I have the runtimes for the default approach=37.2655s, factored approach=8.4112s, optimized approach=1.4129s." I added the stuff in the parentheses.

Table 1: Computational Runtime and Speedup Comparison for 2D Heat Equation Solvers

Linear Algebra Approach	Runtime (seconds)	Speedup Factor
(i) Default (<code>spsolve</code> / Backslash)	37.2655	1.00× (Baseline)
(ii) Factored (Natural, factored as built)	8.4112	4.43×
(iii) Optimized (MMD_AT_PLUS_A, factored w/ sparsity ordering)	1.4129	26.38×

Plotting: My fastest solver was the mode = optimized, which corresponds to the LU factors generated via ‘`spla.splu(G, permc_spec="MMD_AT_PLUS_A")`’. This is the equivalent of matlab’s `symamd` in conjunction with `chol`.

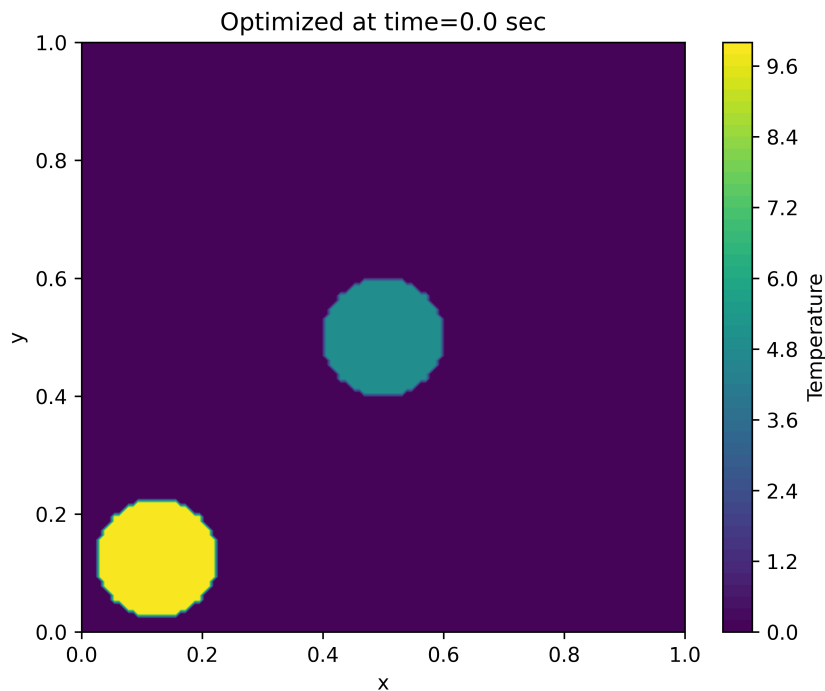


Figure 1: Initial Time, $t=0s$

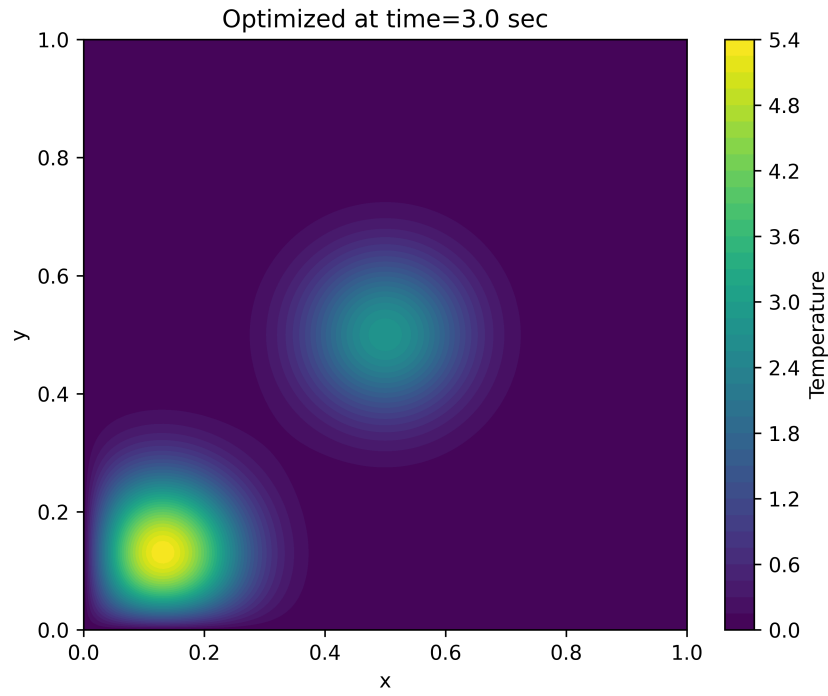


Figure 2: Intermediate Time, $t=3s$

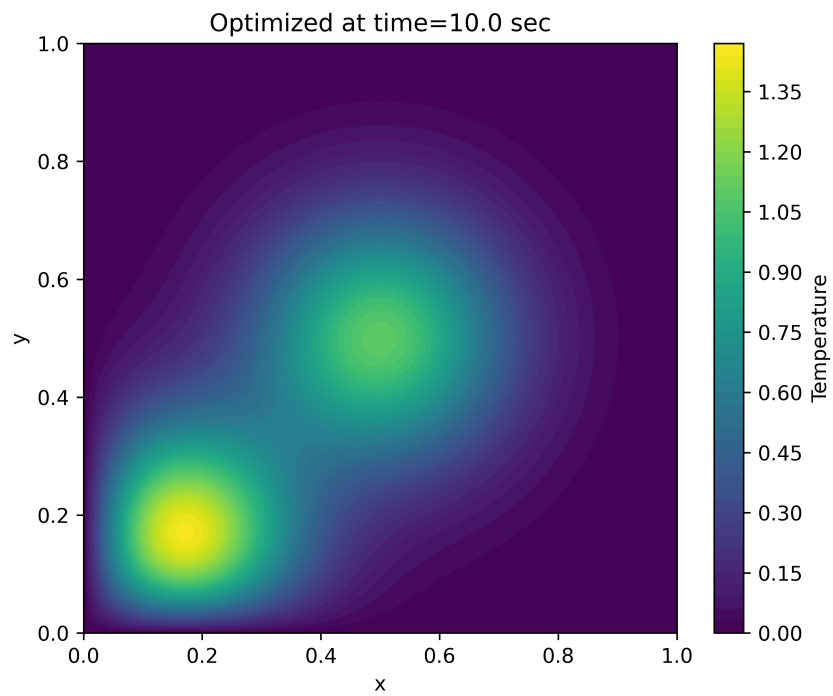


Figure 3: Final Time, $t=10s$

Discussion: From the runtime comparison, we see both factorized methods dramatically outperform the baseline approach where we solve the entire linear system at each step. In particular, pre-computing the LU factors cut the runtime from roughly 37.3s for the default approach to roughly 8.4 for a speedup factor of $4.43\times$. On its own, a $4.43\times$ improvement in runtime is already massive. However, when we use the optimized method where we pre-compute the LU factors and perform sparsity ordering, we see an incredible reduction to roughly 1.4s for a $26.38\times$ speedup factor!!! This performance increase blows both the default and precomputed LU factorization without ordering out of the water. The main takeaway is to exploit sparsity patterns when you can because there are massive performance boosts available if you can.

From the plots, we see exactly what you would expect for a diffusive process. There are two initial regions of higher-than-background (0.0) temperature. In the lower corner, there is a higher temperature region and, in the middle, there is a medium temperature region. Both regions undergo diffusion and eventually the temperature fields merge to form a saddle point around $(x,y)=(0.4,0.4)$. Additionally, we see the high temperature region in the lower left interacting with the boundary condition forming fairly sharp gradients. The boundary condition imposes the temperature equals zero at the boundary. Additionally, we see the second law of thermodynamics in play as the global maximum temperature always decreases as time progresses.

2 Appendix

Listing 1: Question 5 code

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import time

# Difference in generation the AI reported:
# - Slicing (2:end-1 vs 1:-1): Python uses 0-based indexing and exclusive upper
# bounds. The interior nodes 2:end-1 in MATLAB translate to 1:-1 in Python.
# - Meshgrid Orientation (indexing='ij'): Passing indexing='ij' to NumPy's
meshgrid keeps coordinate matrix array structures aligned cleanly with standard
# row/column Cartesian indices.
# - Backslash Operator (\): MATLAB's  $G \setminus U(:,n)$  is handled via
# scipy.sparse.linalg.spsolve(G, U[:, n]) for sparse matrices.

def u0(x, y):
    """Initial_condition_function."""
    term1 = (((x - 0.125)**2 + (y - 0.125)**2) <= 0.01) * 10.0
    term2 = (((x - 0.5)**2 + (y - 0.5)**2) <= 0.01) * 5.0
    return term1 + term2

def get_discrete_laplacian(nx):
    """Computes_discretization_of_Lu=-Laplacian_u."""
    m = nx - 1
    I = sp.eye(m, format='csr')
    e = np.ones(m)
```

```

# Create T and S blocks as sparse matrices
T = sp.diags([-e, 4*e, -e], [-1, 0, 1], shape=(m, m), format='csr')
S = sp.diags([-e, -e], [-1, 1], shape=(m, m), format='csr')

# 2D Laplacian via Kronecker product
A = sp.kron(I, T, format='csr') + sp.kron(S, I, format='csr')
return A

def solve_heat_2D_implicit(mode):
    # I ADDED THIS AND MADE THIS FN OF MODE _____ #
    """
    Parameters
    mode: str
    changes how problem is solved: ["default", "factored", "optimized"]
    """
    # END ADDITION _____ #

    # PDE parameters
    kappa = 0.001

    # Spatial discretization
    nx = 2**7
    h = 1.0 / nx
    xi = np.linspace(0, 1, nx + 1)
    yi = xi
    Xi, Yi = np.meshgrid(xi, yi, indexing='ij')

    # The spatial discretization matrix
    A = get_discrete_laplacian(nx)
    A = kappa * (1.0 / h**2) * A

```

```

# Evaluate initial state and extract interior nodes
U0_full = u0(Xi, Yi)
U0 = U0_full[1:-1, 1:-1]

# Time discretization
t0 = 0
Tf = 10
nt = 500 + 1
dt = Tf / (nt - 1)
ti = np.linspace(t0, Tf, nt)

# Time-stepping setup
num_interior = (nx - 1)**2
# U will store flattened solutions row by row across time steps
U = np.zeros((num_interior, nt))
U[:, 0] = U0.flatten() # Python defaults to row-major flattening
I = sp.eye(num_interior, format='csr')

print(f"Time_integration_in_progress_for_{mode}_approach...")
start_time = time.time()

G = I + dt * A
# Convert to CSR format for efficient row-slicing and solving
G = G.tocsr()

# I ADDED THIS ----- #
# Changing Modes
if mode == "factored":
    # python equivalent of basic 'chol'
    G = G.tocsc() # format splu is more efficient in
    G_lu_factors = splu.splu(G, permc_spec="NATURAL")
if mode == "optimized":

```

```

    # python equivalent of 'symamd'
    G = G.tocsc() # format splu is more efficient in
    G_lu_factors = spla.splu(G, permc_spec="MMD_AT_PLUS_A")
# END ADDITION ----- #

for n in range(nt - 1):
    # Show progress at integer time steps
    if ti[n] % 1 == 0:
        print(f"t_{ti[n]:4.2f}")

    # I ADDED THIS ----- #
    if mode == "default":
        # Solve the linear system (Baseline method: equivalent to backslash \)
        U[:, n + 1] = spla.spsolve(G, U[:, n])
    if mode in ["factored", "optimized"]:
        U[:, n + 1] = G_lu_factors.solve(U[:, n])
    # END ADDITION ----- #

compute_time = time.time() - start_time
print(f"Time_integration_for_{mode}_approach_complete...
~~~~~in_{compute_time:.4f}_seconds\n")

# Store the solution back into a list of 2D grids (including boundary nodes)
U_array = []
for i in range(nt):
    U_mat = np.zeros((nx + 1, nx + 1))
    U_mat[1:-1, 1:-1] = U[:, i].reshape(nx - 1, nx - 1)
    U_array.append(U_mat)

return U_array, ti, Xi, Yi

```

```

if __name__ == "__main__":
    # I ADDED THIS ----- #
    for mode in ["default", "factored", "optimized"]:
        U_array, ti, Xi, Yi = solve_heat_2D_implicit(mode=mode)
        sampled_time = np.arange(0, 11) # creating plots every second

        for sec in sampled_time:
            time_index = int(sec / (10/500))
            time_step = ti[time_index]

            plt.figure(figsize=(6,5))

            contour = plt.contourf(Xi, Yi, U_array[time_index], cmap="viridis", levels=50)
            plt.colorbar(contour, label="Temperature")

            plt.title(f"{mode.capitalize()}_at_time={time_step:.1f}_sec")
            plt.xlabel("x")
            plt.ylabel("y")
            plt.tight_layout()

            file_name=f"hw02q4_{mode}_t{sec:02d}s.png"
            plt.savefig(file_name, dpi=600)
            plt.close()

        print(f"Finished_plotting_for_{mode}_approach")

    print(f"Finished_all_plotting")
    # END ADDITION ----- #

```